TCS302 - DATA STRUCTURES WITH C

Insertion Sort

Algorithm, Implementation and Complexity

Unit IV - Lecture 2

Idea grow a sorted prefix, one element at a time

Dry run every pass traced on 7 3 5 2 9 1

Complexity best O(n), worst and average O(n^2), verified

The idea: sorting a hand of playing cards

Pick up cards one at a time. Each new card is slid into its correct place among the cards already in your hand, which are always kept in order.



- The array is split into a SORTED prefix on the left and an UNSORTED remainder on the right.
- A single element is already sorted, so the prefix starts as just a[0].
- Each pass takes the first unsorted element (the KEY) and inserts it into the prefix.

The algorithm, step by step

1. For i = 1 to n-1, repeat steps 2 to 5.
2. Save the key: key = a[i]. Set j = i - 1.
3. While j >= 0 and a[j] > key: shift a[j] one place right, then j = j - 1.
4. Stop at the first element that is not greater than key (or at the front).
5. Drop the key into the gap: a[j + 1] = key.

Loop invariant

After the pass for index i, the elements a[0..i] are in sorted order. When i reaches n-1, the whole array is sorted. (Checked by an assertion over all 720 orderings of six values.)

Insertion sort in C

```
void insertionSort(int a[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = a[i];
        int j = i - 1;

        while (j >= 0 && a[j] > key)
        {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}
```

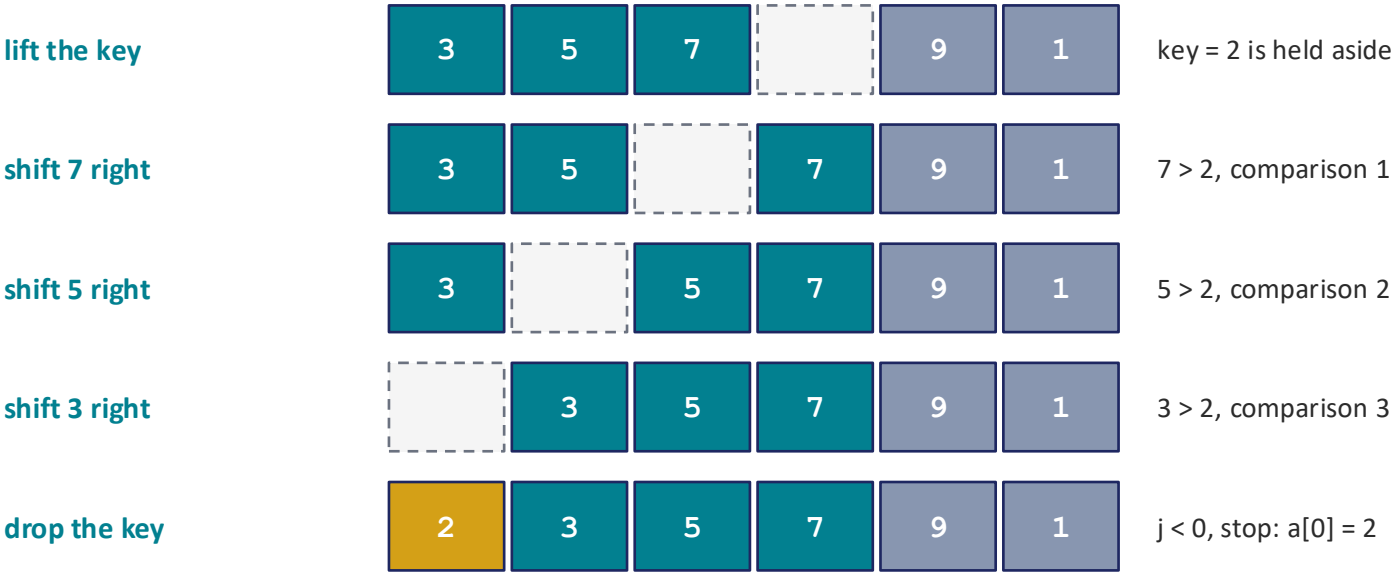
- Nothing is swapped: larger elements are SHIFTED right, which is cheaper than a swap.
- The test a[j] > key is strict, so an equal key is never passed - this is why the sort is stable.
- j >= 0 is tested first, so the loop never reads before the start of the array.

Dry run on 7 3 5 2 9 1 (verified)

Pass	Key	Array after the pass (sorted part | rest)	Comparisons	Shifts
start	-	7 | 3 5 2 9 1	–	–
1	3	3 7 | 5 2 9 1	1	1
2	5	3 5 7 | 2 9 1	2	1
3	2	2 3 5 7 | 9 1	3	3
4	9	2 3 5 7 9 | 1	1	0
5	1	1 2 3 5 7 9	5	5
total		1 2 3 5 7 9	12	10

Pass 4 is cheap because 9 is already larger than everything before it; pass 5 is costly because 1 must travel to the front.

Zoom on pass 3: key = 2 travels to the front



Three shifts and three comparisons - exactly the 3 / 3 recorded for pass 3 in the dry-run table.

Best, worst and average case, measured (n = 8)

Case	Input	Comparisons	Shifts
Best	1 2 3 4 5 6 7 8 (already sorted)	7	0
Nearly sorted	1 2 4 3 5 6 7 8	8	1
Typical	5 2 8 1 9 3 7 4	18	13
Worst	8 7 6 5 4 3 2 1 (reversed)	28	28

- Best case: n-1 = 7 comparisons and no shifts, so insertion sort is O(n) on sorted input.
- Worst case: n(n-1)/2 = 28 comparisons and 28 shifts, so O(n^2).
- Average case: over all 720 orderings of six values the shifts averaged exactly 7.50 = n(n-1)/4, also O(n^2).

Quadratic growth: double the data, quadruple the work

n	comparisons, reversed input	n(n-1)/2	growth from previous n
10	45	45	-
20	190	190	4.22 times
40	780	780	4.11 times

Reversed input
Comparisons match n(n-1)/2 exactly, and doubling n multiplies the cost by about 4 - the signature of O(n^2).

Sorted input, n = 40
Only 39 comparisons and 0 shifts. The same algorithm on the same size of data costs 780 or 39, depending purely on the input.

Properties of insertion sort

Property	Answer	Evidence
Stable	Yes	Strict a[j] > key never passes an equal key (verified in Lecture 1)
In-place	Yes	Only key and j are extra: O(1) memory
Adaptive	Yes	Nearly sorted n = 8 input cost 8 comparisons and 1 shift, not 28
Time, best / average / worst	O(n) / O(n^2) / O(n^2)	Measured on the previous two slides

Simple, stable and fast on small or nearly sorted data, but quadratic on large random data - the motivation for the O(n log n) sorts later in this unit.

UNIT IV - LECTURE 2 - SUMMARY

A sorted prefix that grows by one

- Each pass lifts a[i] as the key, shifts larger elements right, and drops the key into the gap. After pass i, a[0..i] is sorted.
- Dry run on 7 3 5 2 9 1: 12 comparisons and 10 shifts in 5 passes.
- Best case O(n) (7 comparisons for n = 8), worst case O(n^2) (28), average O(n^2) (exact average of 7.50 shifts for n = 6).
- Stable, in-place and adaptive: strong on small or nearly sorted data, weak on large random data.

EXIT TICKET Dry-run insertion sort on 9 4 6 2 5. Show the array after each pass and count the total comparisons and shifts.